ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KĨ THUẬT MÁY TÍNH



BÁO CÁO LAB 3

Môn: Vi xử lý - vi điều khiển(TH)

GVHD: Phan Văn Sỹ

Sinh viên thực hiện : Trần Thanh Hải

MSSV : 2410915

Học kỳ 261, tháng 11 năm 2026

Mục lục

1 Exercise 1 **2**
1.1 Sơ đồ FSM 2

2 Exercise 2 **2**
2.1 Sơ đồ Proteus 2

3 Exercise 3 **3**
3.1 Sơ đồ Proteus 3

4 Exercise 4 **3**
4.1 Code 3

5 Exercise 5 **4**
5.1 Code 4

6 Exercise 6 **4**
6.1 Code 4

7 Exercise 7 **5**
7.1 Code (mode 1) 5
7.2 Code (button 2) 6
7.3 Code (button 3) 6

8 Exercise 8 **7**
8.1 Code 7

9 Exercise 9 **7**
9.1 Code 7

10 Exercise 10 **8**
10.1 Code function 8
10.2 Code main 9

11 Link github **10**

1 Exercise 1

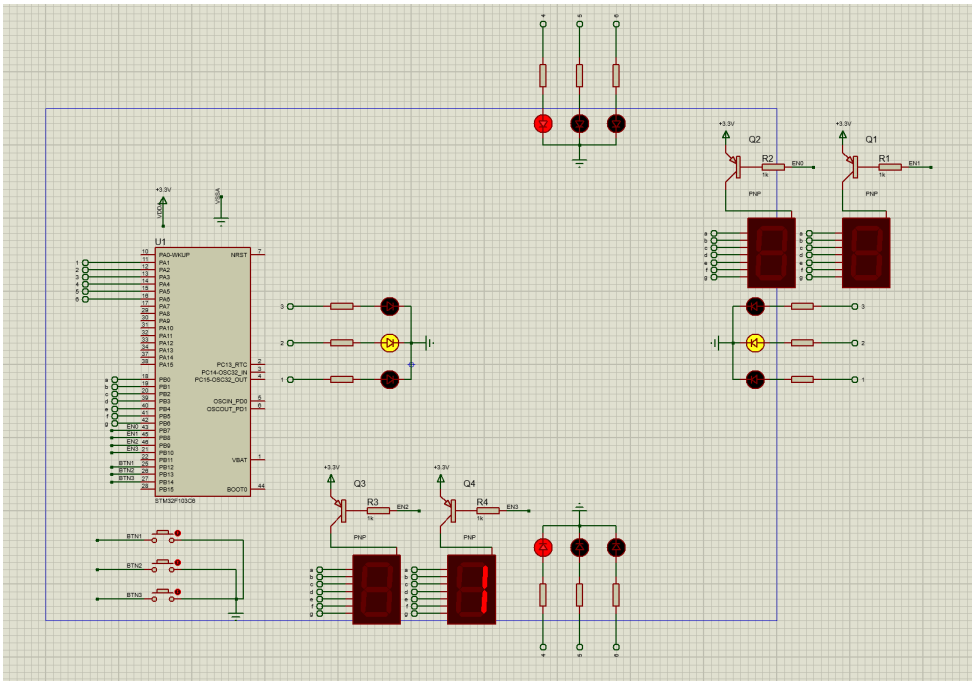
1.1 Sơ đồ FSM

```
[Sơ đồ FSM Mode]:
(Mode 1: Normal) —[Bấm BTN1]—> (Mode 2: Modify Red)
      ▲                                  | [Bấm BTN1]
      | [Bấm BTN1]                       ▼
(Mode 4: Modify Green) <—[Bấm BTN1]— (Mode 3: Modify Yellow)

[Sơ đồ FSM 4 Pha Mode 1]:
(Pha 1: Xanh 1 - Đỏ 2) —[Hết Xanh 1]—> (Pha 2: Vàng 1 - Đỏ 2)
      ▲                                          | [Hết Vàng 1]
      | [Hết Vàng 2]                             ▼
(Pha 4: Đỏ 1 - Vàng 2) <—[Hết Xanh 2]— (Pha 3: Đỏ 1 - Xanh 2)
```

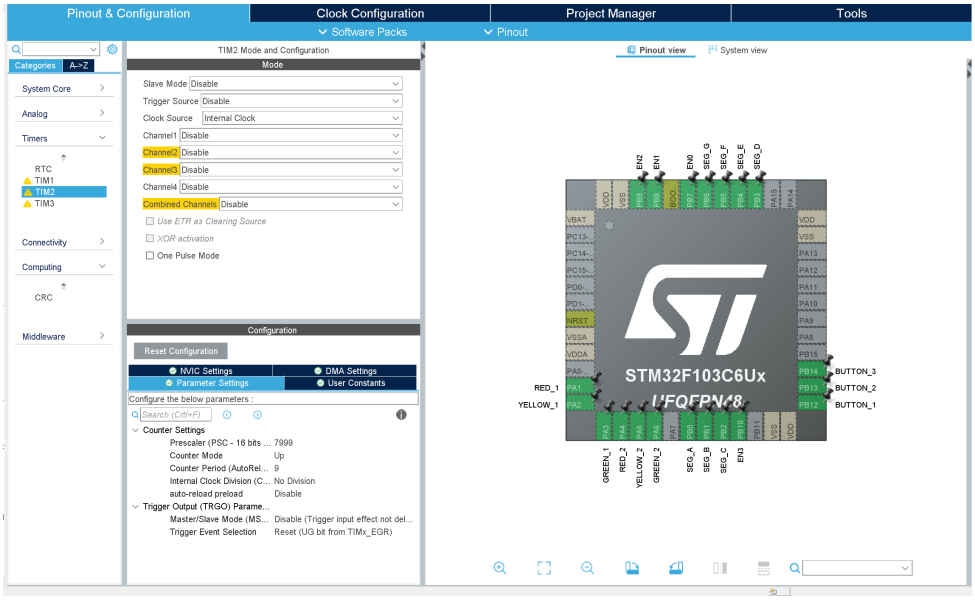
2 Exercise 2

2.1 Sơ đồ Proteus



3 Exercise 3

3.1 Sơ đồ Proteus



4 Exercise 4

4.1 Code

```
// #define TIMER_CYCLE 10

timer_counter_2hz++;
    if (timer_counter_2hz >= (250 / TIMER_CYCLE)) {
      timer_flag_2hz = 1;
      timer_counter_2hz = 0;
    }

    timer_counter_1s++;
    if (timer_counter_1s >= (1000 / TIMER_CYCLE)) {
      timer_flag_1s = 1;
      timer_counter_1s = 0;
    }

    timer_counter_500ms++;
    if (timer_counter_500ms >= (500 / TIMER_CYCLE)) {
      timer_flag_500ms = 1;
      timer_counter_500ms = 0;
    }
```

5 Exercise 5

5.1 Code

```
void button_reading(void) {
  for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
    debounceButtonBuffer2[i] = debounceButtonBuffer1[i];
    debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(BUTTON_PORT[i],
   BUTTON_PIN[i]);

    if (debounceButtonBuffer1[i] == debounceButtonBuffer2[i]) {
      buttonBuffer[i] = debounceButtonBuffer1[i];

      if (buttonBuffer[i] == BUTTON_IS_PRESSED) {
        if (counterForButtonPress1s[i] <
   DURATION_FOR_AUTO_INCREASING) {
          counterForButtonPress1s[i]++;
        } else {
          flagForButtonPress1s[i] = 1;
        }
      } else {
        counterForButtonPress1s[i] = 0;
        flagForButtonPress1s[i] = 0;
      }
    }
  }
}
```

6 Exercise 6

6.1 Code

```
switch (current_mode) {
  case MODE_1_NORMAL:
    runTrafficLightMode1();
    break;

  case MODE_2_MODIFY_RED:
    if (timer_flag_2hz) {
      timer_flag_2hz = 0;
      toggleRedLeds();
    }
    break;

  case MODE_3_MODIFY_YELLOW:
    if (timer_flag_2hz) {
      timer_flag_2hz = 0;
      toggleYellowLeds();
    }
```

```
    break;

  case MODE_4_MODIFY_GREEN:
    if (timer_flag_2hz) {
      timer_flag_2hz = 0;
      toggleGreenLeds();
    }
    break;
  }

\subsection{Code}
\begin{lstlisting}
void update7SEG_scan(void) {

  HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
  HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
  HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
  HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);

  display7SEG(led_buffer[current_led_index]);

  switch (current_led_index) {
  case 0:
    HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET);
    break;
  case 1:
    HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET);
    break;
  case 2:
    HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_RESET);
    break;
  case 3:
    HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_RESET);
    break;
  }

  current_led_index = (current_led_index + 1) % 4;
}
```

7 Exercise 7

7.1 Code (mode 1)

```
    if (current_mode == MODE_1_NORMAL) {
     current_mode = MODE_2_MODIFY_RED;
     temp_time = time_red;
     setLedBuffer(2, temp_time); // Tuyn 1 hin 02, Tuyn 2 hin thi gian
    }
```

7.2 Code (button 2)

```
 // N t 2: T ng  t h i  gian (d ng chung cho Mode 2, 3, 4)
if (current_mode != MODE_1_NORMAL) {
  uint8_t need_update_display = 0;

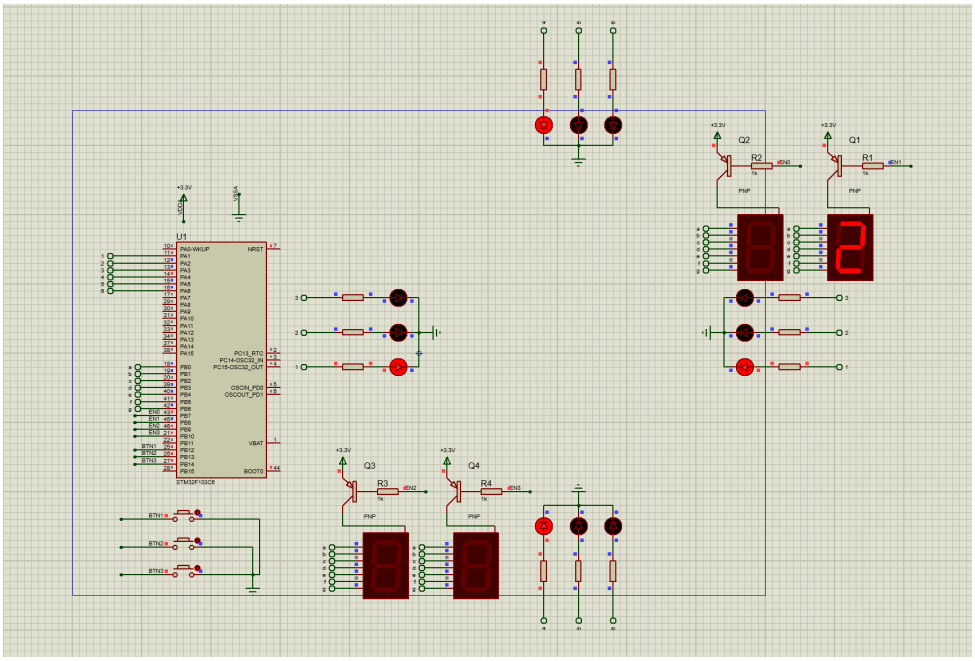
  if (curr_btn2 == 1 && prev_btn2 == 0) {
    temp_time++;
    if (temp_time > 99)
      temp_time = 1;
    need_update_display = 1;
  } else if (is_button_pressed_1s(1) && timer_flag_500ms) {
    timer_flag_500ms = 0;
    temp_time++;
    if (temp_time > 99)
      temp_time = 1;
    need_update_display = 1;
  }

  if (need_update_display) {
    setLedBuffer(current_mode, temp_time);
  }
}
prev_btn2 = curr_btn2;
```

7.3 Code (button 3)

```
 // N t 3: L u  gi   t r   t h i  gian   n
if (curr_btn3 == 1 && prev_btn3 == 0) {
  if (current_mode == MODE_2_MODIFY_RED) {
    time_red = temp_time;
  }
}

// H nh vi trong Mode 2:  N h p  nh y   n             t n
  s   2Hz (250ms)
case MODE_2_MODIFY_RED:
  if (timer_flag_2hz) {
    timer_flag_2hz = 0;
    toggleRedLeds();
  }
  break;
```



8 Exercise 8

8.1 Code

```
if (curr_btn3 == 1 && prev_btn3 == 0) {
  if (current_mode == MODE_3_MODIFY_YELLOW) {
    time_yellow = temp_time;
  }
}
// H nh vi trong Mode 3: N h p nh y n V ng t n
  s 2Hz (250ms)
case MODE_3_MODIFY_YELLOW:
  if (timer_flag_2hz) {
    timer_flag_2hz = 0;
    toggleYellowLeds();
  }
  break;
```

9 Exercise 9

9.1 Code

```
 // N t 3: L u gi t r t h i gian n Xanh
if (curr_btn3 == 1 && prev_btn3 == 0) {
  if (current_mode == MODE_4_MODIFY_GREEN) {
    time_green = temp_time;
  }
}

```

```
  // H nh vi trong Mode 4:  N h p  nh y   n   Xanh      t n   s
    2Hz (250ms)
  case MODE_4_MODIFY_GREEN:
    if (timer_flag_2hz) {
      timer_flag_2hz = 0;
      toggleGreenLeds();
    }
    break;
```

10 Exercise 10

10.1 Code function

```

static void initMode1(void) {
  //  T     ng    c n  b ng    m    b o       = Xanh + V ng
   khi quay  l i  Mode 1
  if (time_red != (time_green + time_yellow)) {
    time_red = time_green + time_yellow;
  }
  current_phase = PHASE_1_G1_R2;
  count_road1 = time_green;
  count_road2 = time_red;
  clearAllTrafficLeds();
  HAL_GPIO_WritePin(GREEN_1_GPIO_Port, GREEN_1_Pin, GPIO_PIN_SET);
  HAL_GPIO_WritePin(RED_2_GPIO_Port, RED_2_Pin, GPIO_PIN_SET);
  setLedBuffer(count_road1, count_road2);
  reset_timer_1s();
}

static void runTrafficLightMode1(void) {
  if (timer_flag_1s) {
    timer_flag_1s = 0;

    count_road1--;
    count_road2--;

    switch (current_phase) {
    case PHASE_1_G1_R2:
      if (count_road1 <= 0) {
        current_phase = PHASE_2_Y1_R2;
        count_road1 = time_yellow;
        clearAllTrafficLeds();
        HAL_GPIO_WritePin(YELLOW_1_GPIO_Port, YELLOW_1_Pin,
   GPIO_PIN_SET);
        HAL_GPIO_WritePin(RED_2_GPIO_Port, RED_2_Pin, GPIO_PIN_SET
   );
      }
      break;

```

```
    case PHASE_2_Y1_R2:
      if (count_road1 <= 0) {
        current_phase = PHASE_3_R1_G2;
        count_road1 = time_red;
        count_road2 = time_green;
        clearAllTrafficLeds();
        HAL_GPIO_WritePin(RED_1_GPIO_Port, RED_1_Pin, GPIO_PIN_SET
  );
        HAL_GPIO_WritePin(GREEN_2_GPIO_Port, GREEN_2_Pin,
  GPIO_PIN_SET);
      }
      break;

    case PHASE_3_R1_G2:
      if (count_road2 <= 0) {
        current_phase = PHASE_4_R1_Y2;
        count_road2 = time_yellow;
        clearAllTrafficLeds();
        HAL_GPIO_WritePin(RED_1_GPIO_Port, RED_1_Pin, GPIO_PIN_SET
  );
        HAL_GPIO_WritePin(YELLOW_2_GPIO_Port, YELLOW_2_Pin,
  GPIO_PIN_SET);
      }
      break;

    case PHASE_4_R1_Y2:
      if (count_road2 <= 0) {
        current_phase = PHASE_1_G1_R2;
        count_road1 = time_green;
        count_road2 = time_red;
        clearAllTrafficLeds();
        HAL_GPIO_WritePin(GREEN_1_GPIO_Port, GREEN_1_Pin,
  GPIO_PIN_SET);
        HAL_GPIO_WritePin(RED_2_GPIO_Port, RED_2_Pin, GPIO_PIN_SET
  );
      }
      break;
    }

    uint8_t disp1 = (count_road1 > 0) ? count_road1 : 0;
    uint8_t disp2 = (count_road2 > 0) ? count_road2 : 0;
    setLedBuffer(disp1, disp2);
  }
}
```

10.2 Code main

```

 int main(void)
{
```

```
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_TIM2_Init();

  /* B t   n g t  Timer 2 chu  k   10ms */
  HAL_TIM_Base_Start_IT(&htim2);

  while (1) {
     /* C h y  m  y  t r ng  th i FSM  x   l   n  t  b m  v
     n   giao th ng */
     fsm_for_input_processing();
  }
}
```

11 Link github

Toàn bộ mã nguồn của các bài thực hành được lưu trữ trên GitHub tại địa chỉ:

- **Repository:** `https://github.com/haitran311/VXL_Lab_3_HCMUT`
- **Nội dung:** Mã nguồn STM32 và các tệp mô phỏng Proteus của các bài thực hành.